Laboratorio 11

En este laboratorio investigue los SMT solvers, algunos de sus usos en la industria, la programacion funcional y la herramienta Z3 de Microsoft. Lo que mas me llamo la atencion es que estos programas permiten convertir reglas de un sistema en formulas y despues buscar una solucion o comprobar que cierta situacion no puede ocurrir. También revise sus limites, porque no todos los problemas se pueden decidir de forma automatica.

Parte 1

a) Que es un SMT

SMT significa Satisfiability Modulo Theories, que se puede traducir como satisfacibilidad modulo teorias. Un solver SMT recibe una formula con varias condiciones y busca valores que hagan verdaderas todas esas condiciones. Si encuentra valores, responde sat. Si las condiciones se contradicen y no existe ninguna solucion, responde unsat. En algunos problemas tambien puede responder unknown, que significa que no logro llegar a una conclusion.

La diferencia principal con SAT es que SAT trabaja con variables que solo pueden ser verdaderas o falsas, mientras que SMT tambien puede trabajar con enteros, numeros reales, cadenas de texto, arreglos, fechas y otras estructuras. Por ejemplo, las condiciones $x > 5$ y $x < 3$ no pueden cumplirse al mismo tiempo. Un solver devuelve unsat porque no existe ningun entero que satisfaga las dos.

b) Como usa AWS los SMT para analizar politicas de seguridad

AWS utiliza una tecnologia llamada Zelkova dentro de servicios como IAM Access Analyzer. Zelkova toma una politica de permisos y la transforma en expresiones logicas. En esas expresiones se representan datos como el usuario o principal, la accion que desea realizar, el recurso y condiciones como una direccion IP.

Despues, los solvers revisan si existe alguna solicitud que la politica permita y que no deberia estar autorizada. Si existe, el solver encuentra un ejemplo concreto y Access Analyzer puede generar una advertencia. Si no existe ninguna solicitud de ese tipo, la formula resulta insatisfacible para ese caso. Esto es diferente de probar manualmente unas cuantas solicitudes, porque el analisis intenta cubrir todas las combinaciones representadas por la politica.

Un ejemplo mostrado por AWS usa una politica con Allow y NotPrincipal. La intencion era excluir a una cuenta, pero en realidad la regla permitia el acceso a casi todos los demas. Zelkova

pudo detectar que existia acceso publico porque comparo la politica con una representacion logica de acceso no autorizado.

c) Tres ejemplos adicionales del uso de SMT

1. Verificacion de hardware. Los circuitos se pueden representar mediante estados y transiciones. El solver busca una secuencia de estados que provoque un error. Si la encuentra, esa secuencia sirve para reproducir y corregir la falla antes de fabricar el componente.

2. Verificacion de software. En la ejecucion simbolica, las entradas de un programa se representan como variables. Cada camino del programa agrega condiciones y el solver intenta encontrar una entrada que cause una division por cero, una asercion incorrecta o una carrera de datos.

3. Contratos inteligentes. Algunas herramientas convierten las reglas de un contrato en formulas y revisan si existe una ejecucion que viole una propiedad, por ejemplo retirar mas dinero del permitido. Esto ayuda a encontrar errores antes de publicar el contrato en una cadena de bloques.

Parte 2

a) Que es la programacion funcional

La programacion funcional es una forma de programar en la que las funciones tienen el papel principal. En lugar de cambiar variables constantemente, se intenta trabajar con datos inmutables y crear nuevos valores a partir de los anteriores. Tambien se usan funciones puras, que producen el mismo resultado cuando reciben la misma entrada y no cambian datos externos.

Este estilo hace que algunas partes del programa sean mas faciles de entender y probar, porque hay menos cambios escondidos en el estado. No significa que todos los lenguajes funcionales prohiban completamente los efectos o la mutacion. Algunos son puramente funcionales y otros permiten combinar varios estilos.

b) Relacion con el calculo lambda y las maquinas de Turing

El calculo lambda es un modelo matematico creado por Alonzo Church. Trabaja principalmente con variables, definicion de funciones y aplicacion de funciones. Muchas ideas que hoy aparecen en lenguajes funcionales vienen de este modelo, como las funciones anonimas, las funciones que reciben otras funciones y la composicion.

Las maquinas de Turing representan la computacion de otra manera: utilizan estados, una cinta y operaciones de lectura y escritura. Aunque los dos modelos son muy diferentes, tienen el mismo poder computacional. Esto quiere decir que una maquina de Turing puede simular un calculo lambda y el calculo lambda puede representar lo que hace una maquina de Turing. Por esa razon, los lenguajes funcionales suficientemente expresivos pueden calcular la misma clase de funciones que otros lenguajes de proposito general.

c) Cinco lenguajes de programacion funcional

1. Haskell: es un lenguaje puramente funcional y usa evaluacion perezosa. Sus funciones se comportan de una forma cercana a las funciones matematicas.

2. Scheme: es un dialecto de Lisp con una sintaxis pequena. Se usa mucho para aprender recursividad, funciones de orden superior y conceptos del calculo lambda.

3. Clojure: funciona sobre la maquina virtual de Java y favorece el uso de datos inmutables. Tambien puede utilizar bibliotecas del ecosistema Java.

4. Erlang: fue pensado para sistemas concurrentes y tolerantes a fallos. Usa procesos aislados que se comunican por medio de mensajes.

5. F#: es un lenguaje de la plataforma .NET. Es multiparadigma, pero permite usar funciones, datos inmutables por defecto y otras tecnicas de programacion funcional.

Parte 3

Z3 de Microsoft

Z3 es un solver SMT y demostrador de teoremas desarrollado por Microsoft Research. Se utiliza para revisar si formulas logicas son satisfacibles. Normalmente no se usa como una aplicacion independiente para usuarios finales, sino como un motor dentro de otras herramientas de verificacion, analisis, planificacion o seguridad.

Para usar Z3 se declaran variables, se agregan restricciones y se pide al solver que las revise. Si responde sat, se puede solicitar un modelo para conocer los valores encontrados. Si responde unsat, no existe una asignacion que cumpla todo. Para demostrar que una propiedad es valida se puede negar la propiedad y pedirle a Z3 que busque un contraejemplo. Si la negacion da unsat, entonces no existe ese contraejemplo.

Programa utilizado

El archivo programa_z3_lab11.py contiene tres pruebas. La primera presenta condiciones contradictorias. La segunda comprueba la validez de una implicacion buscando un contraejemplo. La tercera asigna tres turnos de trabajo respetando varias restricciones.

Para ejecutarlo primero se instala la biblioteca con el comando pip install z3-solver. Despues, desde la carpeta donde se guardo el archivo, se ejecuta python programa_z3_lab11.py. Las capturas se deben tomar despues de ejecutar el programa en la terminal.

```python
def ejemplo_contradiccion():
    print("EJEMPLO 1: CONTRADICCION")
    x = Int("x")
    solver = Solver()
    solver.add(x > 5)
    solver.add(x < 3)

    resultado = solver.check()
    print("Restricciones: x > 5 y x < 3")
    print("Resultado:", resultado)
    if resultado == unsat:
        print("Conclusion: no existe un entero que cumpla las dos condiciones.")
    print()
```

```
EJEMPLO 1: CONTRADICCION
Restricciones: x > 5 y x < 3
Resultado: unsat
Conclusion: no existe un entero que cumpla las dos condiciones.
```

En la primera prueba las condiciones fueron x > 5 y x < 3. El resultado esperado es unsat, porque no existe un numero entero que cumpla las dos condiciones al mismo tiempo.

```python
def ejemplo_validez():
    print("EJEMPLO 2: COMPROBAR UNA IMPLICACION")
    x = Int("x")
    solver = Solver()

    # Para probar que una expresion es valida, Z3 busca un contraejemplo.
    # Si la negacion es insatisfacible, la expresion original es valida.
    propiedad = Implies(x > 5, x > 3)
    solver.add(Not(propiedad))

    resultado = solver.check()
    print("Propiedad: si x > 5, entonces x > 3")
    print("Resultado de buscar un contraejemplo:", resultado)
    if resultado == unsat:
        print("Conclusion: no hay contraejemplo, por lo tanto la propiedad es valida.")
    print()
```

```
EJEMPLO 2: COMPROBAR UNA IMPLICACION
Propiedad: si x > 5, entonces x > 3
Resultado de buscar un contraejemplo: unsat
Conclusion: no hay contraejemplo, por lo tanto la propiedad es valida.
```

En la segunda prueba no se busca directamente un valor para x. Se busca un caso que haga falsa la implicacion. Como Z3 responde unsat, no encontro ningun contraejemplo y la implicacion se considera valida.

```python
def ejemplo_horarios():
    print("EJEMPLO 3: ASIGNACION DE HORARIOS")
    print("Turnos: 0 = manana, 1 = tarde, 2 = noche")

    ana = Int("turno_ana")
    luis = Int("turno_luis")
    marta = Int("turno_marta")

    solver = Solver()

    # Cada persona recibe un turno valido.
    for turno in (ana, luis, marta):
        solver.add(turno >= 0, turno <= 2)

    # No se repiten turnos.
    solver.add(Distinct(ana, luis, marta))

    # Restricciones particulares.
    solver.add(ana != 2)   # Ana no puede trabajar de noche.
    solver.add(luis != 0)  # Luis no puede trabajar por la manana.

    resultado = solver.check()
    print("Resultado:", resultado)

    if resultado == sat:
        modelo = solver.model()
        nombres = {0: "manana", 1: "tarde", 2: "noche"}
        print("Ana:", nombres[modelo[ana].as_long()])
        print("Luis:", nombres[modelo[luis].as_long()])
        print("Marta:", nombres[modelo[marta].as_long()])
        print("Conclusion: Z3 encontro una asignacion que cumple todas las reglas.")
    else:
        print("No existe una asignacion que cumpla todas las reglas.")
    print()
```

```
EJEMPLO 3: ASIGNACION DE HORARIOS
Turnos: 0 = manana, 1 = tarde, 2 = noche
Resultado: sat
Ana: manana
Luis: tarde
Marta: noche
Conclusion: Z3 encontro una asignacion que cumple todas las reglas.
```

La tercera prueba muestra un uso mas practico. Cada persona debe recibir un turno diferente, Ana no puede trabajar de noche y Luis no puede trabajar por la manana. Z3 encuentra una asignacion que cumple todas las reglas y por eso responde sat.

Decidibilidad

Un problema es decidible cuando existe un algoritmo que siempre termina y responde correctamente si o no para cualquier entrada valida. Un problema es indecidible cuando no puede existir un algoritmo con esa garantia para todos los casos. El ejemplo mas conocido es el problema de parada, porque no se puede crear un programa general que determine si cualquier otro programa terminara o se quedara ejecutandose para siempre.

Esto afecta a herramientas como Z3. Hay fragmentos de logica para los que existen procedimientos completos, pero tambien hay combinaciones mas generales que son indecidibles o muy dificiles. Por eso Z3 puede devolver unknown o necesitar un limite de tiempo. Ese resultado no significa que la formula sea falsa; solamente significa que el solver no pudo decidirla con los recursos o metodos disponibles.

Otra herramienta investigada

La otra herramienta que investigue fue cvc5. Tambien es un solver SMT y es de codigo abierto. Puede trabajar con el formato SMT-LIB y con teorias como enteros, numeros reales, cadenas, arreglos y vectores de bits. Se utiliza como motor en herramientas de verificacion de programas, hardware, contratos inteligentes y demostracion de teoremas.

Z3 y cvc5 resuelven problemas parecidos, pero no siempre tienen el mismo rendimiento en todas las formulas. En un proyecto real se puede elegir el solver segun las teorias necesarias, la integracion con el lenguaje de programacion y el tipo de evidencia que se necesita.

Parte 4

Reflexion

Los limites teoricos de la computacion no impiden construir sistemas confiables, pero si obligan a definir bien que se quiere comprobar. En lugar de intentar analizar cualquier propiedad posible de un programa, los ingenieros seleccionan las propiedades mas importantes, como evitar acceso no autorizado o impedir una condicion peligrosa. Despues crean un modelo mas pequeno que pueda ser analizado.

Una prueba formal tampoco significa que todo el sistema real sea perfecto. La prueba se refiere al modelo y a los supuestos que se escribieron. Si el modelo deja afuera una regla importante, el solver no puede inventarla. Por eso estos metodos se combinan con pruebas de software, revision de codigo, monitoreo y controles de seguridad.

Principios o estrategias observadas

1. Trabajar con problemas limitados. Una estrategia es usar fragmentos decidibles, reducir el dominio de los datos o analizar solamente cierta cantidad de pasos. Esto hace posible obtener una respuesta util, aunque la garantia debe explicarse con el alcance utilizado.

2. Buscar contraejemplos. En vez de intentar demostrar una propiedad de forma directa, se niega y se busca una situacion que la viole. Si aparece un contraejemplo, sirve para corregir el sistema. Si no existe dentro del modelo, se obtiene evidencia a favor de la propiedad.

3. Combinar varias tecnicas. Los solvers no reemplazan las pruebas unitarias, la revision humana ni el monitoreo. Cada tecnica encuentra tipos de errores diferentes, por lo que juntas ofrecen mas confianza que una sola herramienta.

Conclusion

Los SMT solvers permiten expresar reglas como formulas y comprobar si existe una solucion o un caso que viole una propiedad. AWS usa esta idea para revisar politicas de acceso y Z3 permite aplicarla a muchos otros problemas. La programacion funcional, el calculo lambda y las maquinas de Turing muestran diferentes formas de representar la computacion, pero tambien recuerdan que existen problemas que ningun algoritmo puede decidir en todos los casos. Por eso lo mas importante es definir bien el problema, entender el alcance del resultado y combinar la verificacion formal con otras tecnicas.

Referencias

Amazon Web Services. How AWS uses automated reasoning to help you achieve security at scale. https://aws.amazon.com/blogs/security/protect-sensitive-data-in-the-cloud-with-automated-reasoning-zelkova/

AWS Identity and Access Management. Understand how IAM Access Analyzer findings work. https://docs.aws.amazon.com/IAM/latest/UserGuide/access-analyzer-concepts.html

Microsoft Research. Online Z3 Guide. https://microsoft.github.io/z3guide/docs/logic/intro/

Microsoft Research. Propositional Logic in Z3. https://microsoft.github.io/z3guide/docs/logic/propositional-logic/

Microsoft Learn. Introduction to Functional Programming Concepts in F Sharp. https://learn.microsoft.com/en-us/dotnet/fsharp/tutorials/functional-programming-concepts

Stanford Encyclopedia of Philosophy. The Lambda Calculus. https://plato.stanford.edu/entries/lambda-calculus/

cvc5 Project. Third Party Applications. https://cvc5.github.io/third-party-applications.html